



山东大学

信息科学与工程学院

2024—2025 学年第二学期

实 验 报 告

课程名称: 微处理器原理与应用

实验名称: 实验 1.2

掌握 Debug 的基本命令及其功能

专 业 班 级 通信工程 二班

学 生 学 号 202400120045

学 生 姓 名 刘哲旭

实 验 时 间 2025 年 3 月 11 日

实验报告

【实验目的】

1.掌握 DEBUG 的基本命令及其功能，掌握 win11 使用 DEBUG 功能

【实验要求】

1.熟练掌握 DEBUG 工具，能够实时查看 CPU 内部各寄存器的当前值、内存单元的存储内容，并以机器码级别对程序执行过程进行单步跟踪和观察。。

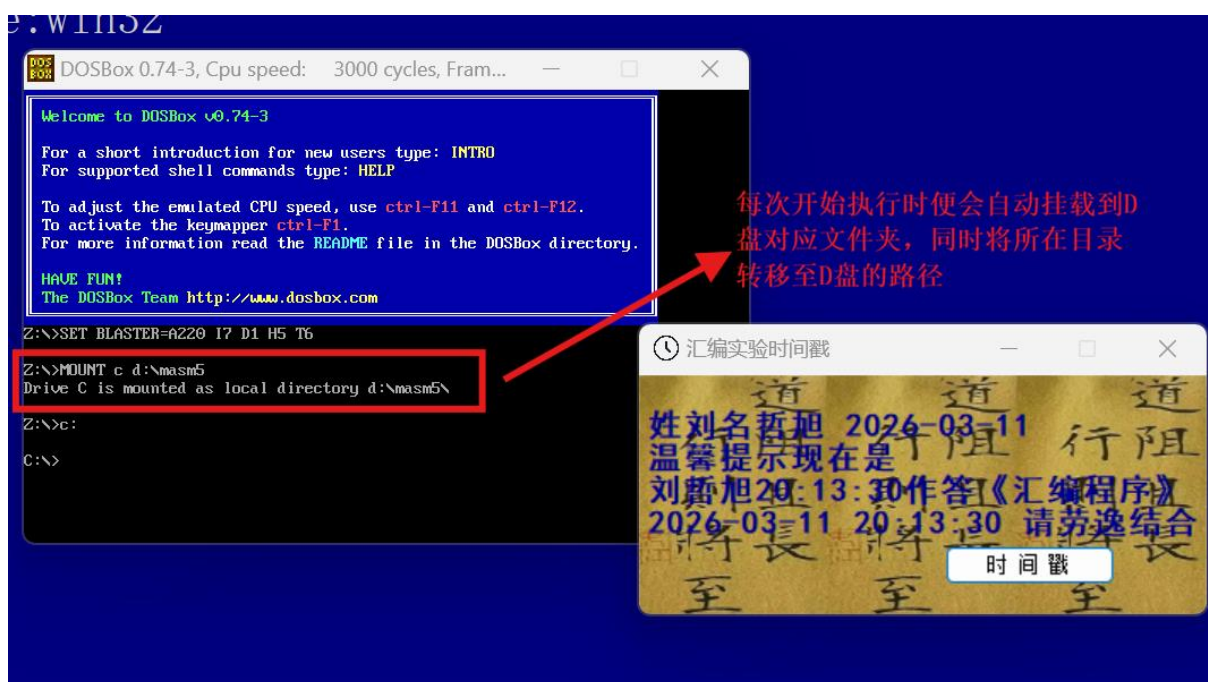
【实验具体内容】

【第一个实验】

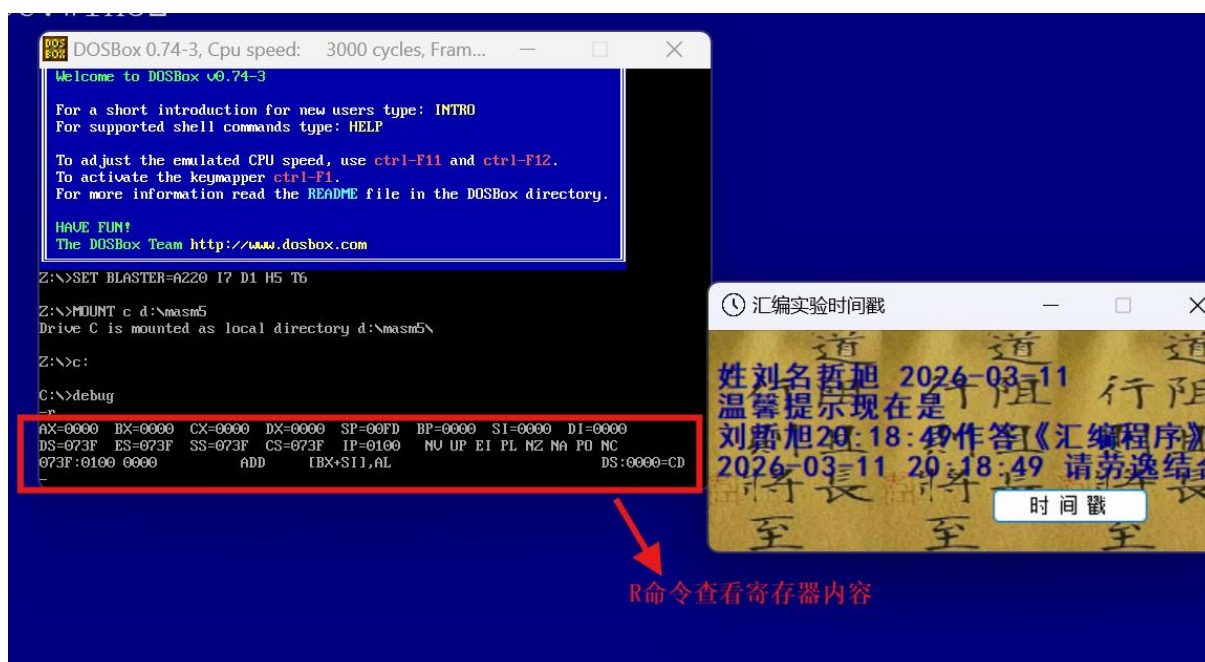
一、实验步骤

(1) 使用 Debug，将要求的程序段逐条写入内存并执行，观察每条指令执行后，CPU 中相关寄存器中内容的变化：

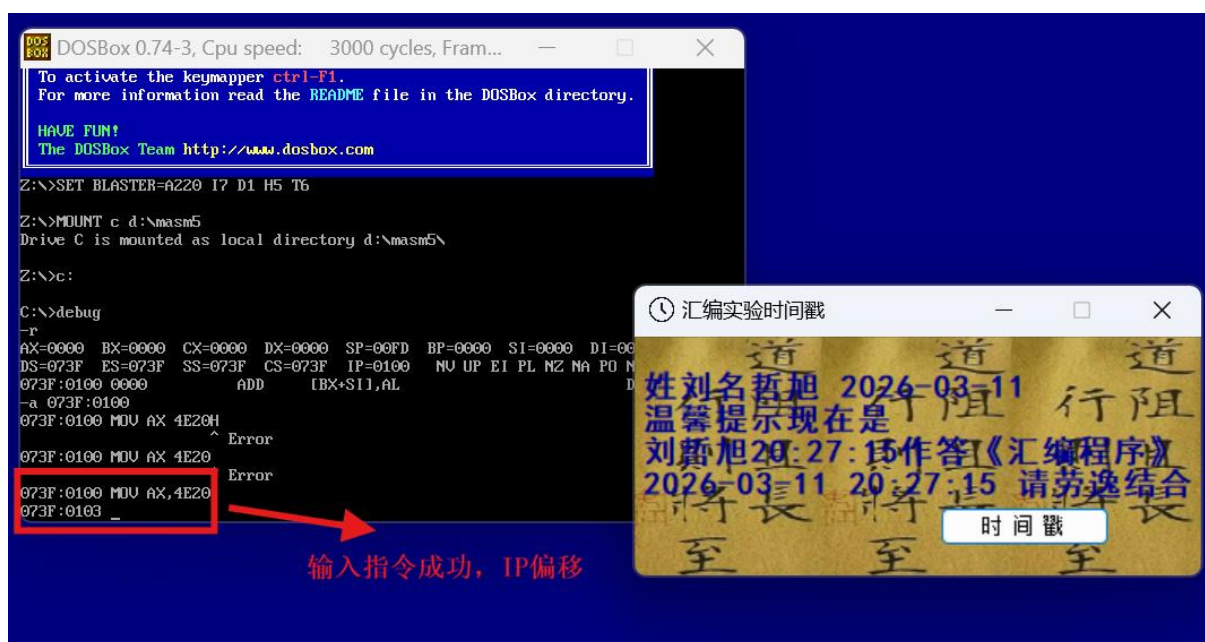
①DOSBox 启动时会自动把挂载到 D 盘对应的主机文件夹，并直接把当前工作目录切换到 D 盘路径下，如下图所示。



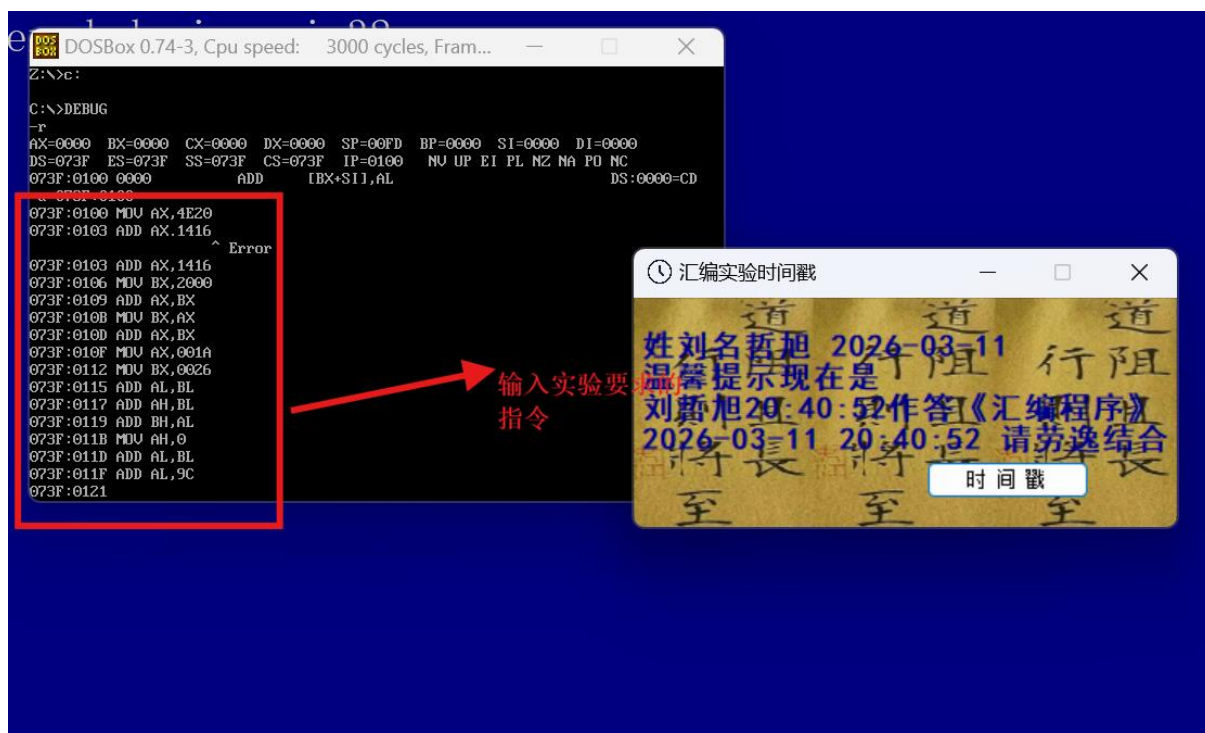
②在 debug 环境中执行-r 命令查看 CPU 寄存器内容，可见 AX、BX、CX、DX 寄存器的值均为 0000H 此时代码段寄存器 CS = 073FH，指令指针 IP = 0100H，对应的 20 位物理地址为 074F0H，该地址存放的指令为 ADD[BX+SI], AL，如下图所示



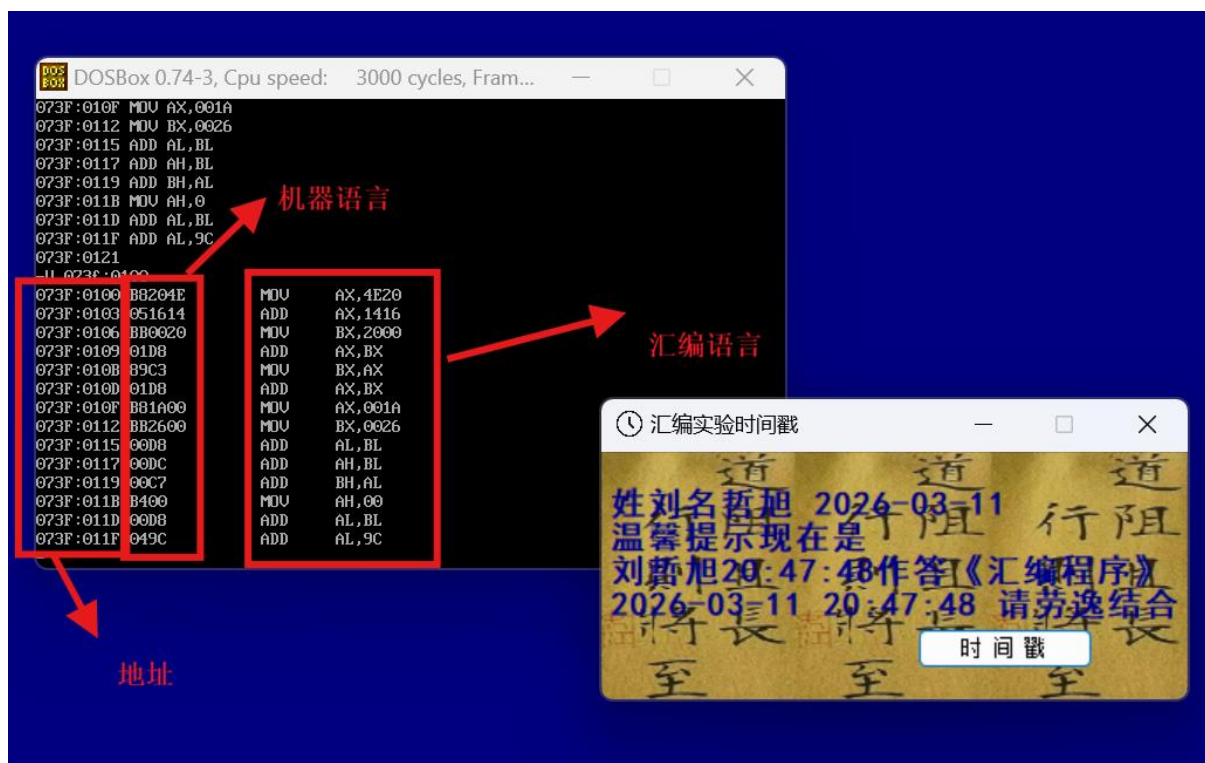
③因为当前 CS:IP 是 073F:0100，所以使用 -a 073F:0100 命令进入汇编模式，在该地址输入 MOV AX,4E20 后按回车，如下图所示：



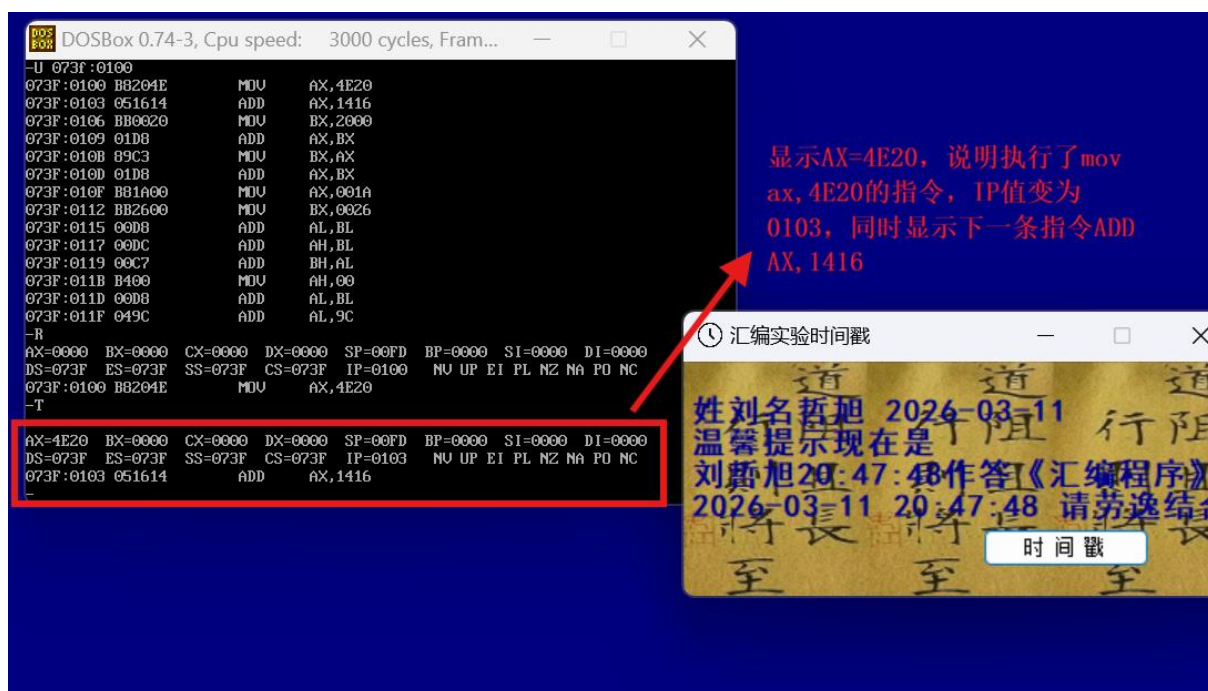
④继续输入实验要求的全部指令，如下图所示：



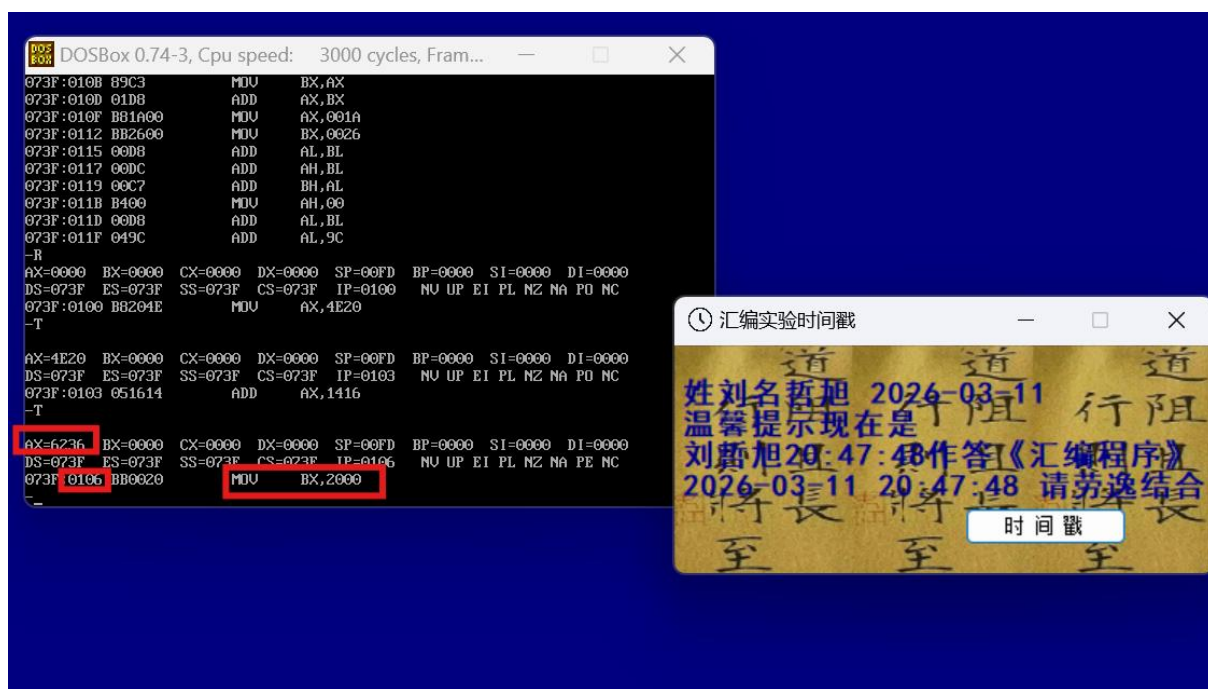
⑤输入指令-u 073F:0100 并按回车，即可查看刚才写入的指令。显示结果中：从左到右分别是内存地址，对应的机器码和汇编指令形式，如下图所示：



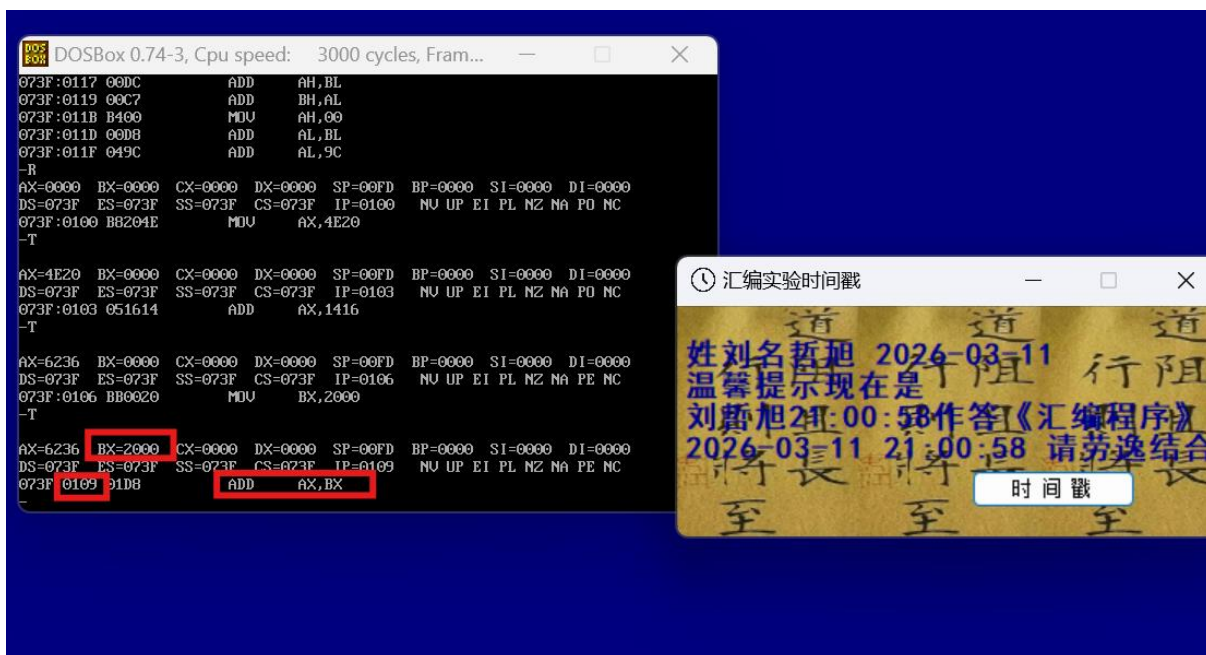
⑥在 debug 指令区输入 -t (单步执行)，执行第一条汇编指令后：寄存器 AX 的值由 0000H 变为 4E20H，IP 值变为 0103H，CS:IP 指向下一条指令 ADD AX,1416 如下图所示：



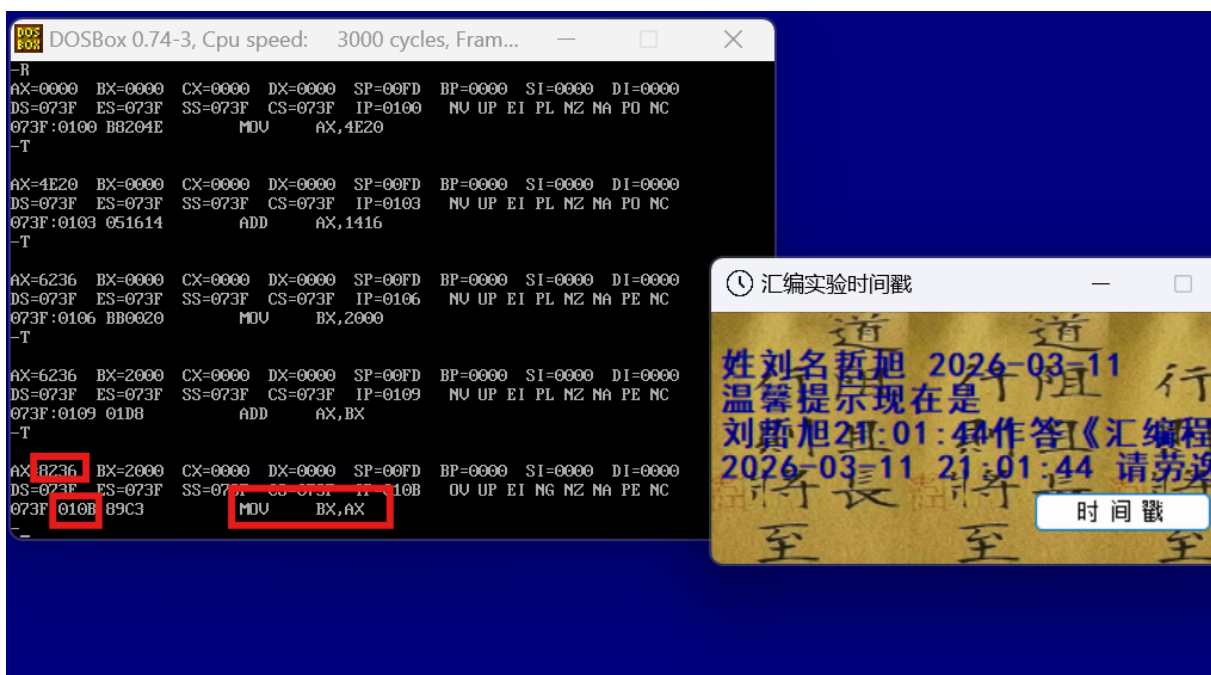
继续在 debug 指令区输入 -t, 逐条单步执行后续指令, 寄存器变化如下:
 执行第 2 条 (ADD AX,1416): AX 从 4E20H → 6236H, IP → 0103H, 下一条: MOV BX,2000



继续在 debug 指令区输入 -t, 逐条单步执行后续指令, 寄存器变化如下:
 执行第 3 条 (MOV BX,2000): BX 从 0000H → 2000H, IP → 0109H, 下一条: ADD AX,BX



继续在 debug 指令区输入 -t, 逐条单步执行后续指令, 寄存器变化如下:
 执行第 4 条(ADD AX,BX): AX 从 6236H → 8236H, IP → 010BH, 下一条: MOV BX,AX



继续在 debug 指令区输入 -t, 逐条单步执行后续指令, 寄存器变化如下:
 执行第 5 条(MOV BX,AX): BX 从 2000H → 8236H, IP → 010DH, 下一条: ADD AX,BX

```

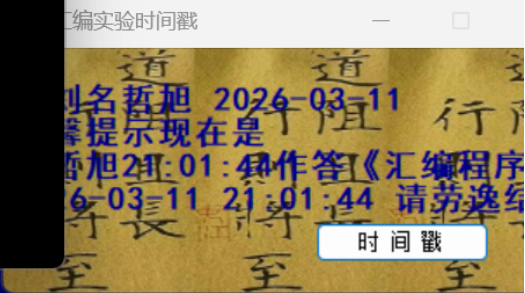
DOSBox 0.74-3, Cpu speed: 3000 cycles, Fram...
AX=4E20 BX=0000 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=073F IP=0103 NU UP EI PL NZ NA PO NC
073F:0103 051614 ADD AX,1416
-T
AX=6236 BX=0000 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=073F IP=0106 NU UP EI PL NZ NA PE NC
073F:0106 BB0020 MOV BX,2000
-T
AX=6236 BX=2000 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=073F IP=0109 NU UP EI PL NZ NA PE NC
073F:0109 01D8 ADD AX,BX
-T
AX=8236 BX=2000 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=073F IP=010B DU UP EI NG NZ NA PE NC
073F:010B 89C3 MOV BX,AX
-T
AX=8236 BX=8236 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=073F IP=010D DU UP EI NG NZ NA PE NC
073F:010D 01D8 ADD AX,BX
-s
    
```

继续在 debug 指令区输入 -t, 逐条单步执行后续指令, 寄存器变化如下:
 执行第 6 条 (ADD AX,BX): AX 从 8236H → 046CH, IP → 010FH, 下一条: MOV AX,001A

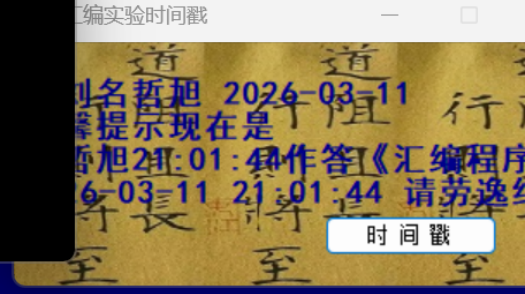
```

DOSBox 0.74-3, Cpu speed: 3000 cycles, Fram...
AX=6236 BX=0000 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=073F IP=0106 NU UP EI PL NZ NA PE NC
073F:0106 BB0020 MOV BX,2000
-T
AX=6236 BX=2000 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=073F IP=0109 NU UP EI PL NZ NA PE NC
073F:0109 01D8 ADD AX,BX
-T
AX=8236 BX=2000 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=073F IP=010B DU UP EI NG NZ NA PE NC
073F:010B 89C3 MOV BX,AX
-T
AX=8236 BX=8236 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=073F IP=010D DU UP EI NG NZ NA PE NC
073F:010D 01D8 ADD AX,BX
-T
AX=046C BX=8236 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=073F IP=010F DU UP EI PL NZ NA PE CY
073F:010F 81A00 MOV AX,001A
-s
    
```

继续在 debug 指令区输入 -t, 逐条单步执行后续指令, 寄存器变化如下:
 执行第 7 条 (MOV AX,001A): AX 从 046CH → 001AH, IP → 0112H, 下一条: MOV BX,0026



执行第 8 条(MOV BX,0026):BX 从 8236H → 0026H,IP → 0115H,下一条:ADD AL,BL



执行第 9 条 (ADD AL,BL): AL 从 1AH $\rightarrow$ 40H, IP $\rightarrow$ 0117H, 下一条: ADD AH,BL

```

DOSBox 0.74-3, Cpu speed: 3000 cycles, Fram...
AX=8236 BX=8236 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=073F IP=010D 00 UP EI NG NZ NA PE NC
073F:010D 01D8 ADD AX,BX
-T
AX=046C BX=8236 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=073F IP=010F 00 UP EI PL NZ NA PE CY
073F:010F B81A00 MOV AX,001A
-T
AX=001A BX=8236 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=073F IP=0112 00 UP EI PL NZ NA PE CY
073F:0112 B82600 MOV BX,0026
-T
AX=001A BX=0026 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=073F IP=0115 00 UP EI PL NZ NA PE CY
073F:0115 00D8 ADD AL,BL
-T
AX=0040 BX=0026 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=073F IP=0117 00 UP EI PL NZ AC PO NC
073F:0117 90DC ADD AH,BL
-T

```

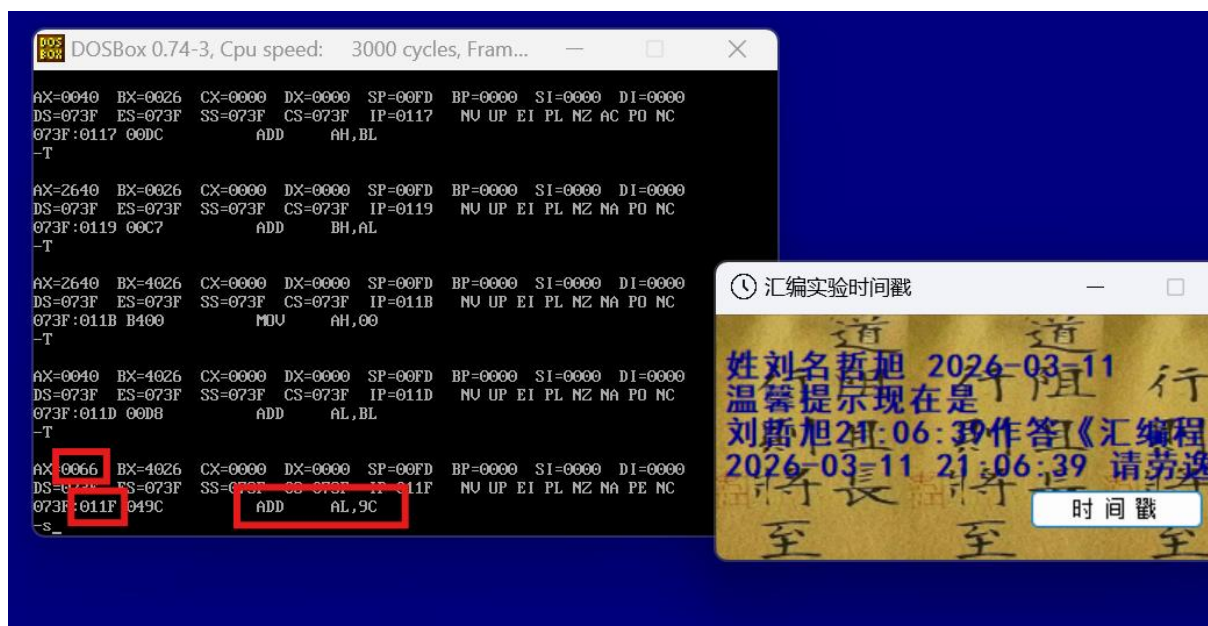
继续在 debug 指令区输入 -t, 逐条单步执行后续指令, 寄存器变化如下:
 执行第 10 条 (ADD AH,BL): AH 从 00H → 26H, IP → 0119H, 下一条: ADD BH,AL

```

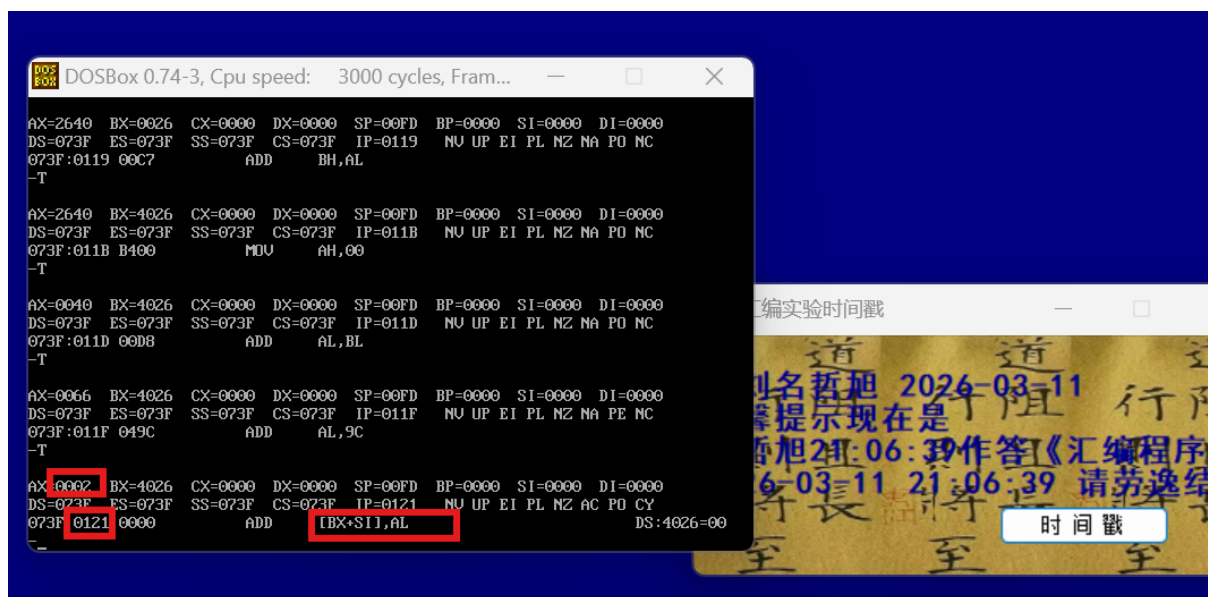
DOSBox 0.74-3, Cpu speed: 3000 cycles, Fram...
AX=046C BX=8236 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=073F IP=010F 00 UP EI PL NZ NA PE CY
073F:010F B81A00 MOV AX,001A
-T
AX=001A BX=8236 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=073F IP=0112 00 UP EI PL NZ NA PE CY
073F:0112 B82600 MOV BX,0026
-T
AX=001A BX=0026 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=073F IP=0115 00 UP EI PL NZ NA PE CY
073F:0115 00D8 ADD AL,BL
-T
AX=0040 BX=0026 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=073F IP=0117 00 UP EI PL NZ AC PO NC
073F:0117 90DC ADD AH,BL
-T
AX=2640 BX=0026 CX=0000 DX=0000 SP=00FD BP=0000 SI=0000 DI=0000
DS=073F ES=073F SS=073F CS=073F IP=0119 00 UP EI PL NZ NA PO NC
073F:0119 90C7 ADD BH,AL
-T

```

继续在 debug 指令区输入 -t, 逐条单步执行后续指令, 寄存器变化如下:
 执行第 11 条 (ADD BH,AL): BH 从 00H → 10H, IP → 011BH, 下一条: MOV AH,00

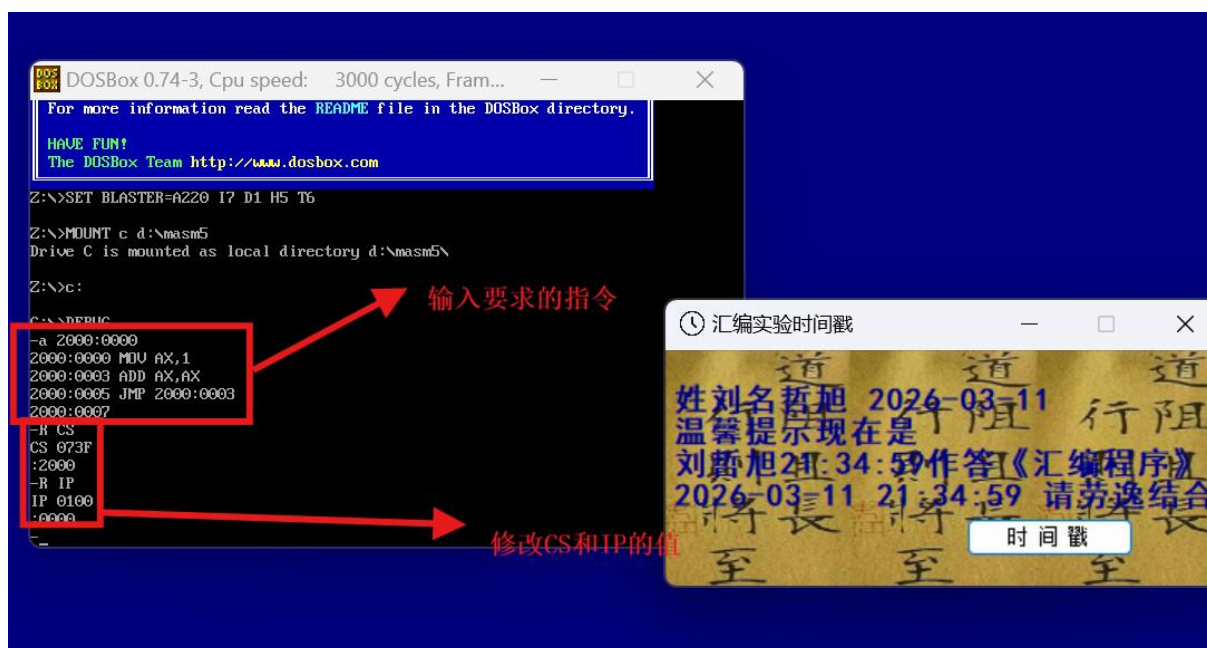


继续在 debug 指令区输入 -t，逐条单步执行后续指令，寄存器变化如下：
 执行第 14 条(ADD AL,9C): AL 从 66H → 02H, IP → 011FH, 下一条: ADD [BX+SI],AL



(2) 将实验要求的三条指令写入从 2000: 0 开始的内存单元中，利用这三条指令计算 2 的 8 次方

①在 debug 指令区输入 -a 2000: 0 指令，依次输入以下汇编指令: mov ax,1、add ax,ax、jmp 2000: 3，之后输入 -r 命令修改 CS: IP，使其指向地址 2000: 0000



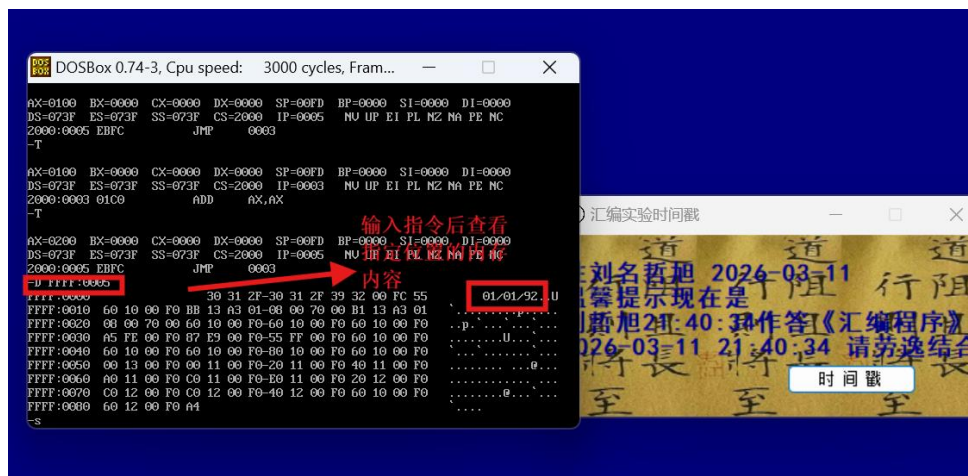
②连续多次输入 -t 单步执行, 观察 AX 寄存器的值逐步变化, 经过总共 16 次执行后, AX 中得到 2 的 8 次方 (即 256), AX=0100 如图所示:



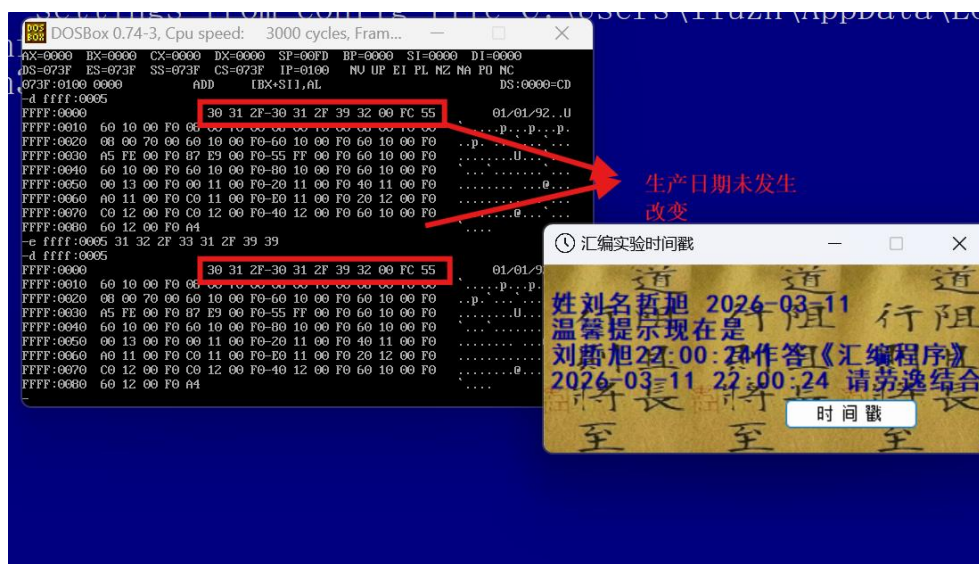


(3) 查看内存中的内容

①输入指令-d ffff:0005，右侧的内存转译后是 01/01/92，可能是月/日/年的格式，可以判断是日期，如下图所示。



②输入指令-e ffff:0005 31 32 2F 33 31 2F 39 39 修改生产日期为 12/31/99，按下回车后发现生产日期并未发生改变，如下图所示：



查阅资料得知，debug 是 DOS 和 Windows 系统中提供的实模式（8086 模式）程序调试工具。然而，在 Windows 11 中，系统已不再自带 debug 工具。虽然可以通过安装 DOSBox 软件来模拟 DOS 环境，但其内存中的日期信息是虚拟生成的，因此不能作为实际参考。此外，尝试修改“生产日期”失败的原因是内存地址 ffff:0005~ffff:000C 属于 ROM（只读存储器），其内容无法被修改。

③在 windows 的 DOS 环境下使用 wmic bios get releasedate

在 Windows 中按 Win + R 打开“运行”对话框，输入 cmd 回车进入命令提示符（DOS 环境）。先执行 wmic bios get description，查看输出得知这是联想主板的当前 BIOS 版本号。进一步查阅 WMIC 相关文档后，使用 wmic bios get releasedate 命令获取 BIOS 的发布日期

```

C:\Windows\system32\cmd.exe
Microsoft Windows [版本 10.0.22631.6199]
(c) Microsoft Corporation。保留所有权利。

C:\Users\liuzh>wmic bios get description
Description
1.10

C:\Users\liuzh>wmic bios get releasedate
ReleaseDate
20251021000000.000000+000

C:\Users\liuzh>
    
```

【第二个实验】

【下篇】用机器指令和汇编指令编程

一、实验步骤

(1) 使用 debug 将实验要求的程序段写入内存，逐条执行，根据指令执行后的实际运行情况填空。

①通过 -r 命令初始化 CS 和 IP 寄存器，将 CS:IP 设置为 1000:0000，使其指向内存地址 1000:0，如下图所示：

```

DOSBox 0.74-3, Cpu speed: 3000 cycles, Fram...
For a short introduction for new users type: INTRO
For supported shell commands type: HELP

To adjust the emulated CPU speed, use ctrl-F11 and ctrl-F12.
To activate the keymapper ctrl-F1.
For more information read the README file in the DOSBox directory.

HAVE FUN!
The DOSBox Team http://www.dosbox.com

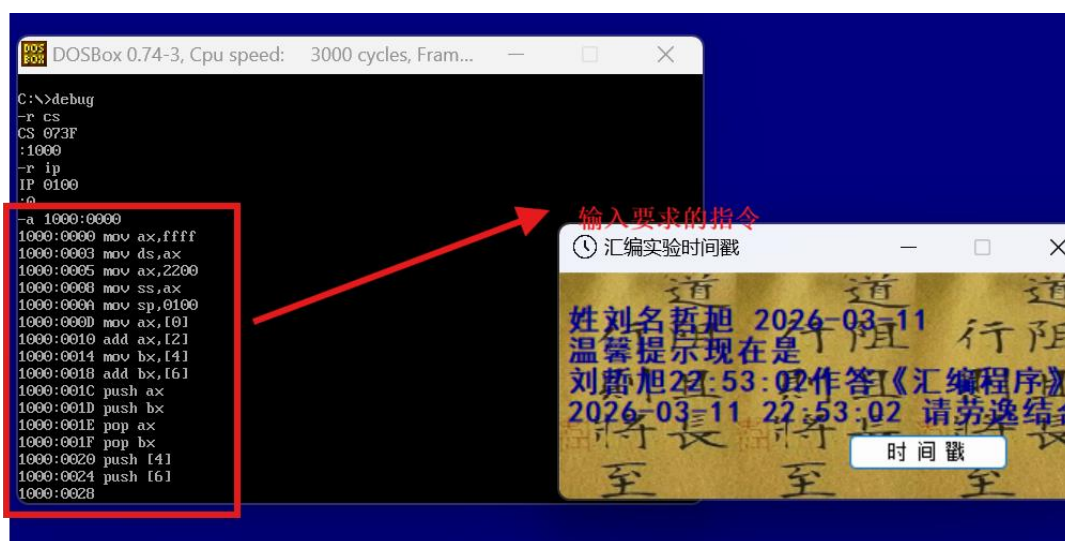
Z:\>SET BLASTER=A220 I7 D1 H5 T6

Z:\>MOUNT c d:\masm5
Drive C is mounted as local directory d:\masm5\

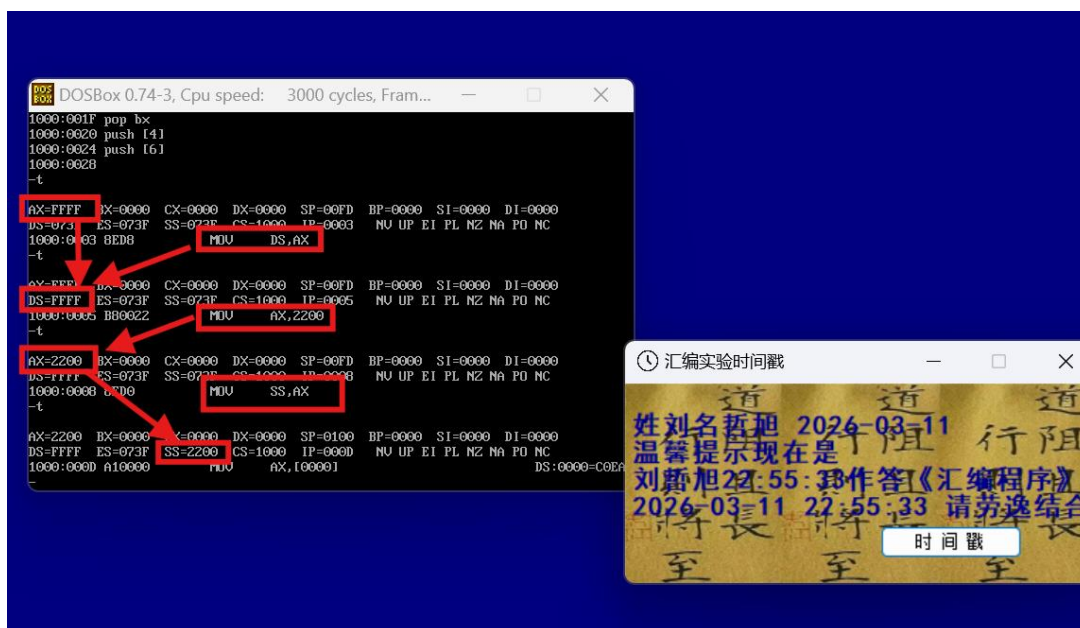
Z:\>c:

C:\>debug
-r cs
CS 073F
:1000
-r ip
IP 0100
:0
    
```

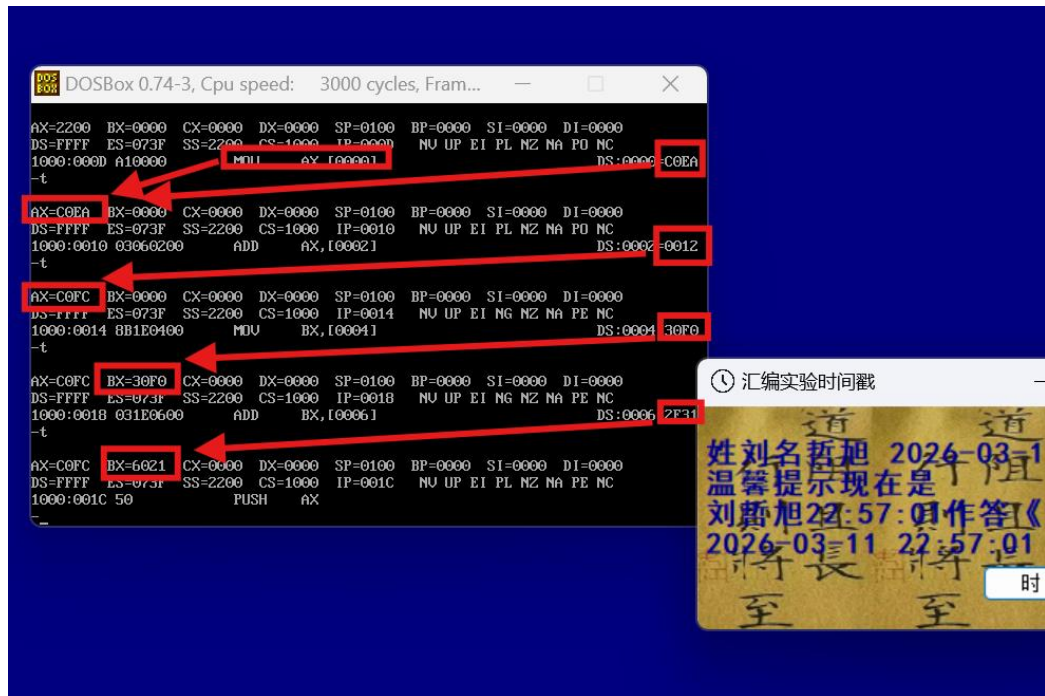
②随后执行 -a 1000:0000 进入汇编模式，依次输入以下指令：MOV AX,FFFF; MOV DS,AX; MOV AX,2200，如下图所示：



③输入指令 `-t` 单步执行：第 1 次执行：AX 被赋值为 FFFFH。第 2 次执行将寄存器 AX 的值送入段寄存器 DS（即 $DS \leftarrow FFFFH$ ），第三次执行：AX 再被赋值为 2200H，之后第四次执行 SS 也被赋值为 2200H。



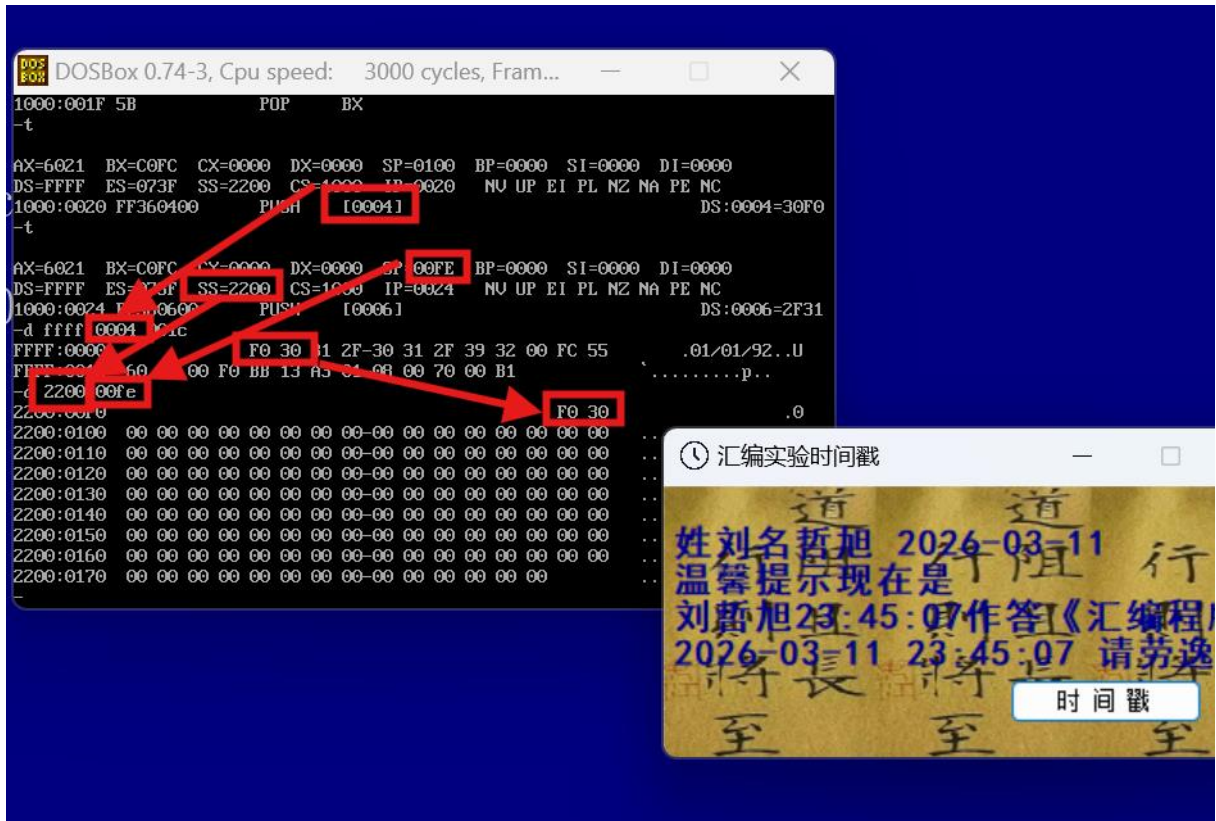
④继续输入指令 `-t` 单步执行：DS: SS 指向地址 0000 的数值 C0EA 赋值给 AX，再从 DS:0002 读出 0012 加到 AX 上；BX 也是一样，先被赋 30F0，再加上 2F31，如下图所示



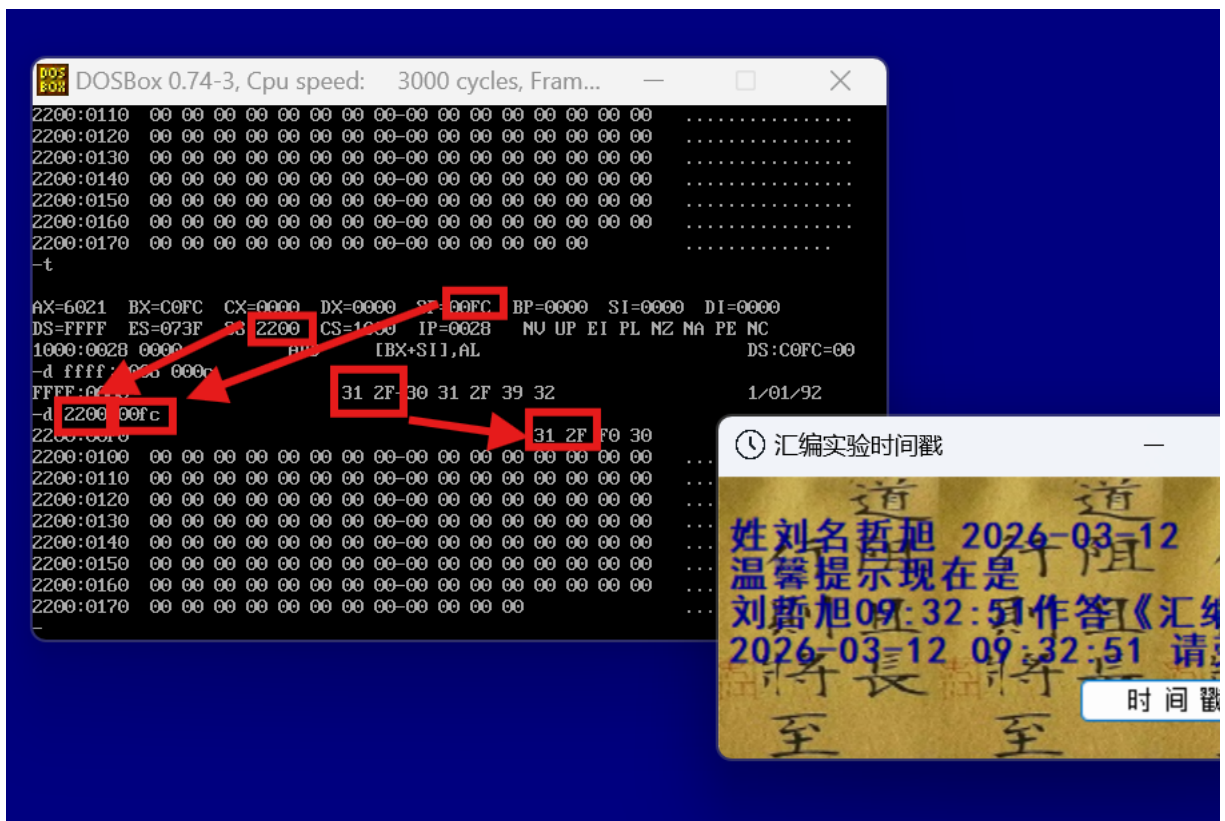
⑤接下来，将 AX 寄存器中的当前值压入栈中，栈指针 SP 相应减小到 00FEH。紧接着压入 BX 的值后，SP 进一步减小至 00FCH。随后执行 POP AX 和 POP BX 两条出栈指令，SP 重新恢复到栈顶位置 0100H。经过这一系列压栈和出栈操作，AX 和 BX 寄存器中的值实现了相互交换。如下图所示



⑥最后将 FFFF:0004 处的数据压入栈中，输入指令-d ss:sp 查看栈顶内容，可见数据已正确入栈，如下图所示，对应的内容为 F0 30



ffff: 6 的数据同理，如下图所示，对应的内容为 31 2F

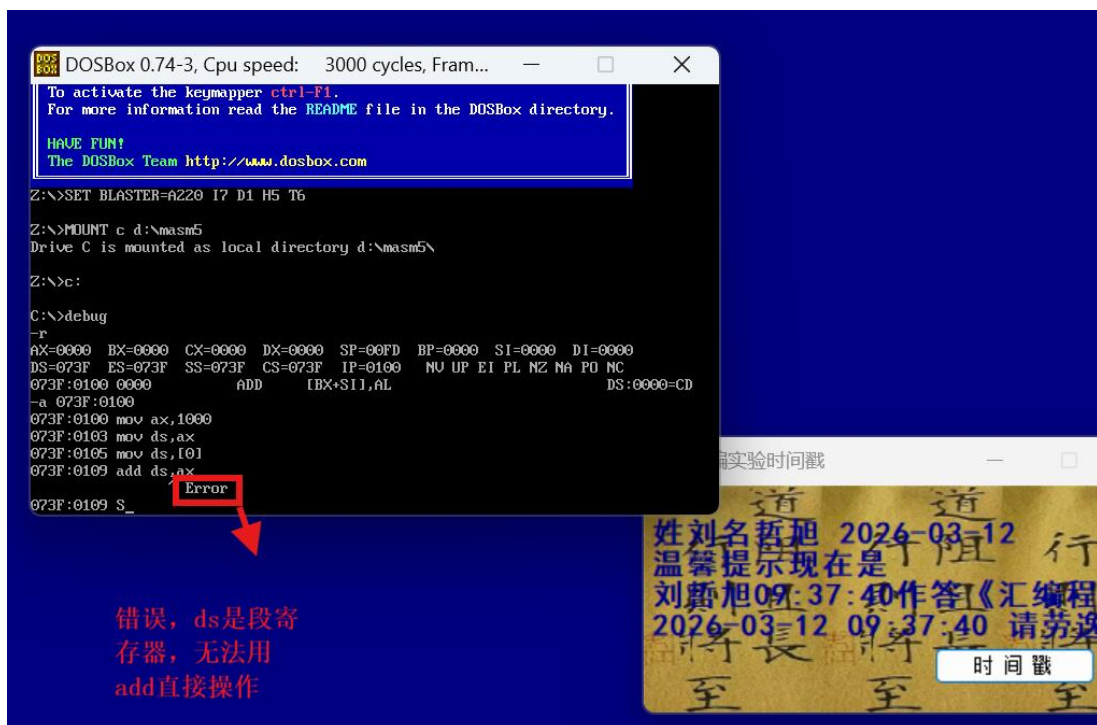


⑦根据上述实验进行填空，如下图所示：

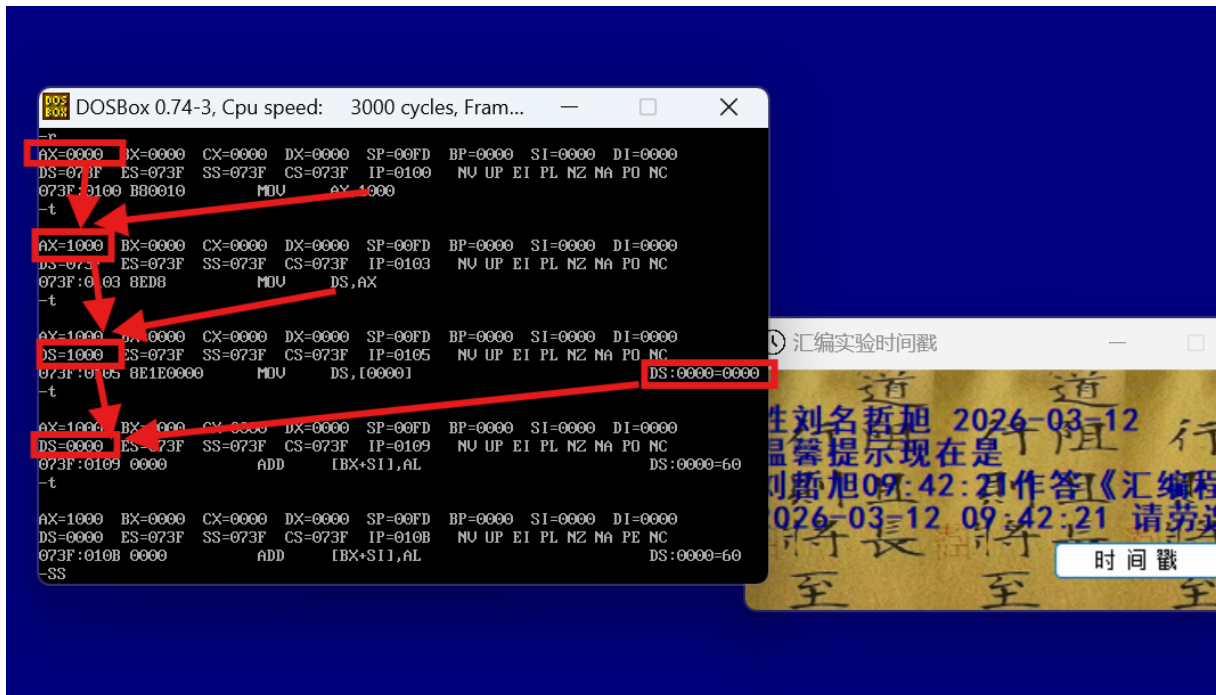
mov ax,ffff	
mov ds,ax	
mov ax,2200	
mov ss,ax	
mov sp,0100	
mov ax,[0]	; ax= <u>COFA</u>
add ax,[2]	; ax= <u>COFC</u>
mov bx,[4]	; bx= <u>30FO</u>
add bx,[6]	; bx= <u>6021</u>
push ax	; sp= <u>00FE</u> ; 修改的内存单元的地址是 <u>2200:00FE-2200:00FF</u>
内容为 <u>FC C0</u>	
push bx	; sp= <u>00FC</u> ; 修改的内存单元的地址是 <u>2200:00FC-2200:00FD</u>
内容为 <u>21 60</u>	
pop ax	; sp= <u>00FE</u> ; ax= <u>6021</u>
pop bx	; sp= <u>0100</u> ; bx= <u>COFC</u>
push [4]	; sp= <u>00FE</u> ; 修改的内存单元的地址是 <u>2200:00FE-2200:00FF</u>
内容为 <u>F0 30</u>	
push [6]	; sp= <u>00FC</u> ; 修改的内存单元的地址是 <u>2200:00FC-2200:00FD</u>
内容为 <u>31 2F</u>	

(2) 使用 debug, 将给定的程序段写入内存, 逐行执行, 观察每条程序执行后, CPU 中相关寄存器中内容的变化

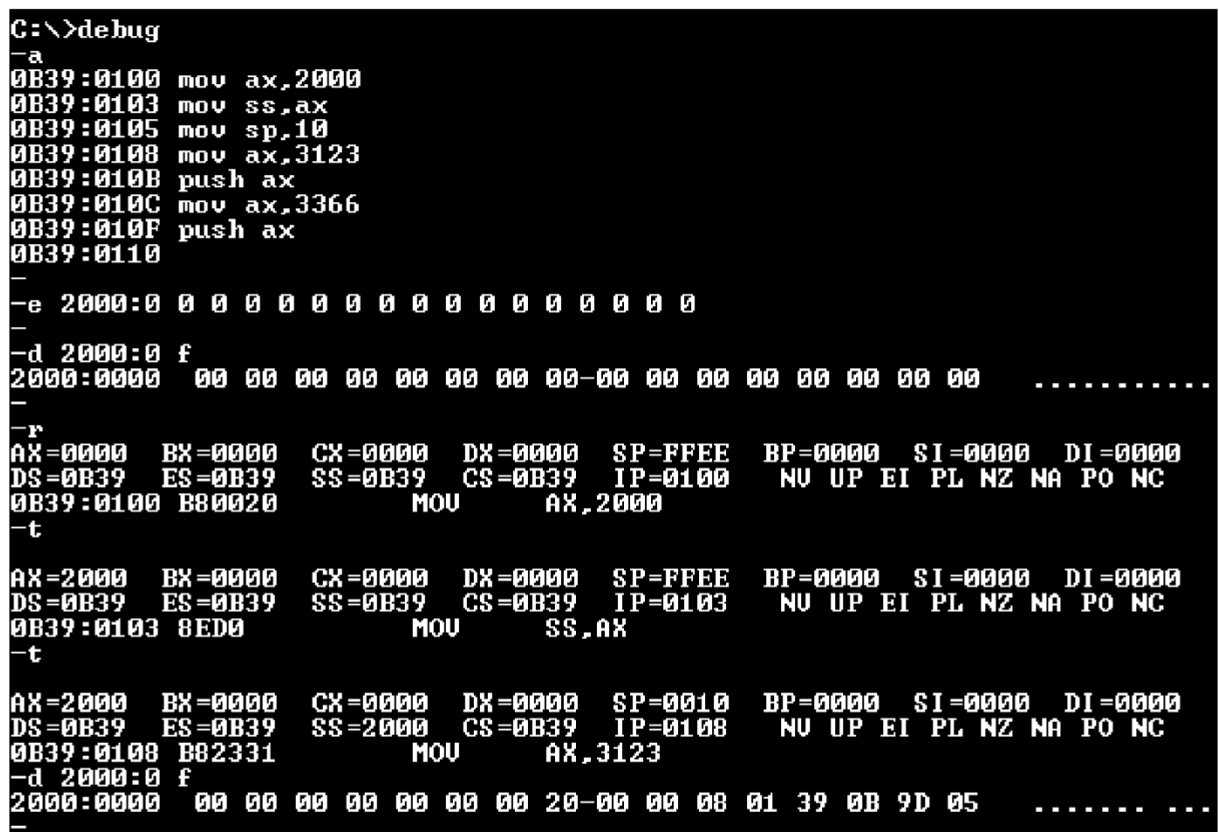
①在 debug 中输入-a 命令开始编写程序, 依次输入各条汇编指令。程序输入到最后一行时, 尝试写入 ADD DS,AX, 结果出现错误, 无法完成该指令的录入。错误原因在于: DS 是段寄存器, ADD 指令不支持直接对其进行算术运算。



②输入-t 单步执行，指令依次执行指令，第一步执行 AX 先被赋值为 1000，第二次执行将 Ax 的值存入 DS，第三步执行后将 1000:0000 地址的 0000 存入 DS，如下图所示



(3) 仔细观察下图中的实验过程，然后分析：为什么 2000:0~2000:f 中的内容会发生改变？



答：深入分析 `-t` 指令的实现原理，可以看出它本质上依赖 CPU 的单步中断机制（类型 01H）。

当执行 `-t` 时，`debug` 会设置单步标志（`TF=1`），从而使 CPU 在每条指令执行完毕后自动触发单步中断。

在中断处理流程中，CPU 会自动把当前 `FLAGS`、`CS`、`IP` 等关键寄存器压入栈中，以便中断返回时恢复现场。

正是这一自动压栈操作，导致栈段 `2000:0000~2000:000F` 这块内存区域的内容被改写，因为它被用作保存中断现场的临时存储空间。

这一现象清晰地体现了 CPU 中断处理的底层逻辑：为了保证被中断的程序能正确继续执行，CPU 必须在栈上保存现场，而栈就成了关键寄存器值的临时存储区域。

【实验心得】

1.汇编平台输入数字时不能加 H，我在输入的时候因为在数字后面加了 H，导致按下回车键后报错；与此同时，这些汇编指令中需要用逗号隔开（例如 `MOV AX,2000`），在实验过程中因为疏忽大意，忘记输入逗号导致结果报错。

2.在完成实验计算 2 的 8 次方中，我是先输入 `-a 2000: 0` 指令，依次输入以下汇编指令：`mov ax,1`、`add ax,ax`、`jmp 2000: 3`，之后输入 `-r` 命令修改 `CS: IP`，使其指向地址 `2000: 0000`。我仔细想了想，如果调换一下顺序，也就是先输入 `-r` 命令修改 `CS: IP`，使其指向地址 `2000: 0000`。再输入 `-a 2000: 0` 指令，依次输入以下汇编指令：`mov ax,1`、`add ax,ax`、`jmp 2000: 3`。尝试过后发现结果一样，依然可以计算 2 的 8 次方。

经过查询资料得：`debug` 的 `-a 2000:0` 命令强制指定了汇编的起始地址是 `2000:0000`，它不参考当前 `CS:IP` 的值。所以无论你先把 `CS:IP` 改成 `2000:0000` 再写程序，还是先写程序再改 `CS:IP`，三条指令最终都会被正确写入物理地址 `20000h` 开始的内存单元。

3.关于实验下篇的第二个：使用 `debug`，将给定的程序段写入内存，逐行执行，观察每条程序执行后，CPU 中相关寄存器中内容的变化：

我发现在执行 `add ds,ax` 的指令时报错，尝试多次均无果。后查阅资料得知在 8086/8088 汇编语言中，`DS`（Data Segment，数据段寄存器）不能直接用 `ADD` 指令进行运算。我还想进一步了解背后的原因以及其他寄存器的情况。我继续询问 AI，AI 给出答复如下：

首先，8086 的寄存器可以分为 **通用寄存器**（`AX`、`BX`、`CX`、`DX` 等）和 **段寄存器**（`CS`、`DS`、`SS`、`ES`）。通用寄存器主要用于算术运算和数据处理，因此可以直接参

与 ADD、SUB 等算术指令。而段寄存器的作用是**存放段地址，用于形成物理地址**，它们主要参与地址计算，而不是一般的数据运算。由于段寄存器具有特殊功能，CPU 的指令系统规定**算术逻辑指令不能直接对段寄存器操作**，因此像 ADD DS,AX 或 ADD DS,1 这样的指令在语法上是非法的。

如果确实需要对 DS 的值进行调整，一般需要**借助通用寄存器进行间接操作**。具体做法是：先把 DS 的值送到一个通用寄存器中进行运算，然后再把结果送回 DS。例如

```
MOV AX, DS
```

```
ADD AX, 10H
```

```
MOV DS, AX
```

【课堂补充】

问题一：小明有一天配了个电脑，他没有装 windows10 或者 windows11 操作系统 装了 Windows XP，老板要卖给他一个 16g 的内存，小明是否该买？

答：Windows XP 系统（包括其 Service Pack 3 版本）对物理内存的支持存在严格限制：32 位版本实际可识别和使用的内存上限通常在 3.2GB 至 3.5GB 之间（受限于 4GB 地址空间和系统保留部分），64 位版本虽理论支持更高容量，但实际应用极少且兼容性差。即使安装 16GB 内存条，操作系统最多仅能有效利用约 3.2GB，其余容量将完全无法被识别和使用，造成显著资源浪费。此外，在某些主板和 BIOS 配置下，安装远超系统支持容量的内存模块还可能引发兼容性问题，例如系统启动失败、黑屏或设备识别异常。

问题二：8086 微处理器内部结构分为两个主要功能部件：左边的执行部件和 右边的总线接口部件，这两大部分的功能是什么？包含哪些组件？

答：**执行部件（EU）**的主要功能是负责指令的**译码和实际执行**，它不直接连接外部总线，所有数据进出都通过 BIU 中转。EU 的核心工作包括：接收 BIU 预取并排队的指令、进行译码、产生控制信号、完成算术/逻辑运算、数据传送以及更新标志位等。EU 包含的主要组件有：

1. 算术逻辑单元（ALU）：执行加、减、与、或、异或、移位等 16 位或 8 位运算。
2. 通用寄存器组：AX、BX、CX、DX（每个可拆分为高低 8 位，如 AH/AL）。
3. 指针和变址寄存器：SP（栈指针）、BP（基指针）、SI（源变址）、DI（目的变址）。
4. 标志寄存器（FLAGS）：存储运算结果的状态标志，如进位、零、符号、溢出等。
5. 指令译码器和控制电路：负责指令分析并协调 EU 内部操作。

总线接口部件（BIU）的主要功能是负责 CPU 与外部存储器、I/O 设备之间的**所有总线操作**，包括产生 20 位物理地址、取指令、读/写数据、地址计算以及指令预取排队。它使 EU 能专注于计算，而无需直接管理外部总线。BIU 包含的主要组件有：

1. 4 个 16 位段寄存器：CS（代码段）、DS（数据段）、SS（堆栈段）、ES（附加段），用于段基址。

2. 指令指针 IP（16 位）：指向当前代码段中的下一条指令偏移。
3. 20 位地址加法器：实现“段基址 $\times 16$ + 偏移”的物理地址计算。
4. 指令队列（Instruction Queue）：8086 为 6 字节（8088 为 4 字节），支持指令预取（当 EU 执行时，BIU 提前从内存取后续指令存入队列，按 FIFO 原则供给 EU）。
5. 总线控制逻辑：产生读/写控制信号、管理总线周期。



知识点

1.debug 相关指令总结

(1) **-r (Register)**：显示或修改 CPU 寄存器的内容。

不带参数：显示所有寄存器（AX、BX、CX、DX、SP、BP、SI、DI、DS、ES、SS、CS、IP）和标志位（FLAGS）。

带参数如 `-r ax`：只显示或允许修改 AX 的值。常用于设置初始 CS:IP（如 `-r cs`、`-r ip`）或检查运行状态。

(2) **-d (Dump)**：以十六进制格式显示指定内存区域的内容。

格式：`-d [段:偏移] [长度]` 或 `-d 段:起始偏移 结束偏移`

示例：`-d 1000:0 f`（显示 1000:0000 到 1000:000F 的 16 个字节）。

最常用查看内存数据、栈内容、程序代码的机器码等。

(3) **-e (Enter)**：修改指定内存单元的内容（逐字节输入）。

格式：`-e 段:偏移 值1 值2 ...`（可连续输入多个字节，按空格分隔，按回车结束）。

示例：`-e 2000:0 00 00 01 02`（把 2000:0000 开始的 4 个字节改为 00 00 01 02）。

常用于清零内存、填充测试数据或手工修改机器码。

(4) **-u (Unassemble)**：反汇编（disassemble）指定内存区域的机器码，显示成可读的汇编指令。

格式：`-u [段:偏移] [长度]` 或 `-u 段:起始偏移 结束偏移`

示例：`-u 1000:0 10`（反汇编从 1000:0000 开始的 16 字节）。

这是检查程序是否正确写入内存、查看机器码对应指令的最常用命令。

(5) **-t (Trace)**：单步执行一条指令（Trace one instruction）。

不带参数：执行当前 CS:IP 指向的一条指令，然后停止并显示所有寄存器变化。

带参数如 `-t 5`：连续单步执行 5 条指令。

核心调试手段，用于观察每条指令执行后的寄存器、标志位、内存变化，常结合 `-r` 和 `-d` 使用。

(6) **-a (Assemble)**：进入汇编输入模式，从指定地址开始逐行输入汇编指令，debug 会自动转换为机器码写入内存。

格式：**-a [段:偏移]**（不带地址则从当前 CS:IP 开始）。

示例：**-a 2000:0**，然后输入 **mov ax,1、add ax,ax、jmp 2000:3**，回车结束。

用于直接在内存中编写/修改小程序，是汇编实验中最常用的命令。

2.PUSH 和 POP

在 8086 汇编语言中，**PUSH** 和 **POP** 是用于栈操作的最基本指令，它们共同实现了数据的入栈和出栈功能。栈是一个后进先出（LIFO）结构，由 **SS** 寄存器指定栈段基址，**SP** 寄存器始终指向当前栈顶（栈中最新的有效数据所在的偏移地址）。8086 的栈向下生长，即从高地址向低地址扩展，每次 **PUSH** 或 **POP** 操作均以字（16 位）为单位进行。

PUSH 指令（压栈）用于将数据压入栈顶，其执行过程为：首先将 **SP** 减 2，为新数据腾出空间，然后将源操作数（可以是寄存器、段寄存器、内存字或立即数）的 16 位值写入 **SS:SP** 指向的内存单元，低字节存放在较低地址，高字节存放在较高地址（小端序存储）。**PUSH** 不影响任何标志位，常用于保存寄存器现场、传递子程序参数或在中断和调用时由硬件自动保存返回地址。

POP 指令（弹栈）用于从栈顶弹出数据，其执行过程为：首先将 **SS:SP** 指向的内存单元中的 16 位值读出赋给目的操作数，然后将 **SP** 加 2，使栈顶指针上移，释放空间。**POP** 同样不影响标志位（除 **POPF** 外），常用于恢复寄存器现场或从栈中取出子程序参数。需要注意的是，**POP** 的目的操作数不能是 **CS** 寄存器，因为这会破坏程序的执行流。**PUSH** 和 **POP** 必须严格配对使用：每次 **PUSH** 对应一次 **POP**，否则会导致栈失衡

3.WMIC

WMIC (Windows Management Instrumentation Command-line) 是 Windows 系统内置的一个命令行工具，用于通过命令行方式访问和操作 **Windows Management Instrumentation (WMI)** 系统。它允许管理员查询系统硬件、软件、进程、服务、用户账户、BIOS、网络配置等信息，甚至执行某些管理操作，如终止进程或调用方法。

常见 **WMIC** 命令如下：

BIOS：查询 BIOS 信息 示例：

```
wmic bios get manufacturer,name,version,serialnumber,smbiosbiosversion /format:list
```

（常用于获取主板 BIOS 厂商、版本、序列号和发布日期）

OS：操作系统信息示例：

```
wmic os get caption,version,buildnumber,installDate,serialnumber,lastbootuptime
```

/format:list （显示 Windows 版本、构建号、安装日期、序列号、上次启动时间）

CPU: 处理器信息 示例:

```
wmic cpu get name,manufacturer,processorid,cores,currentclockspeed,maxclockspeed
/format:list
```

COMPUTERSYSTEM: 计算机系统概览（主机名、域、登录用户、型号等） 示例:

```
wmic computersystem get name,domain,username,model,manufacturer
```

PROCESS: 运行进程示例:

```
wmic process get name,processid,executablepath,commandline /format:list 终止进
程: wmic process where name="notepad.exe" delete
```

SERVICE: 系统服务 示例:

```
wmic service get name,displayname,state,startmode /format:list 启动服务: wmic
service where name="spooler" call startservice
```

PRODUCT: 已安装软件（Add/Remove Programs） 示例:

```
wmic product get name,version,vendor,installDate /format:list
```

QFE: 已安装的 Windows 更新（补丁） 示例:

```
wmic qfe get description,hotfixid,installedon
```

LOGICALDISK: 磁盘分区信息示例:

```
wmic logicaldisk get name,description,filesystem,size,freespace /format:list
```

NICCONFIG: 网络适配器配置（IP、MAC 等） 示例:

```
wmic nicconfig get
description,ipaddress,macaddress,defaultipgateway,dnsserversearchorder
```

USERACCOUNT: 用户账户 示例: wmic useraccount get name,sid,status

STARTUP: 开机启动项 示例: wmic startup get caption,command,user

4. 在 DOS 的 DEBUG 命令中:

举例: -d ffff:0004 表示: 从内存地址 FFFF:0004 开始显示内存内容。

如果没有写第二个参数, DEBUG 会默认显示 128 个字节 (即 80H 字节) 的数据, 并按行列出。-d ffff:0004 001c 和 -d ffff:0004 相比, 指定了显示范围, 显示从 ffff:0004-ffff:001c 的内存内容, 共 29 个字节

```

DOS
BOX
DOSBox 0.74-3, Cpu speed: 3000 cycles, Fram...
HAVE FUN!
The DOSBox Team http://www.dosbox.com

Z:\>SET BLASTER=A220 I7 D1 H5 T6

Z:\>MOUNT c d:\masm5
Drive C is mounted as local directory d:\masm5\

Z:\>c:

C:\>debug
-d ffff:0004
FFFF:0000          F0 30 31 2F-30 31 2F 39 32 00 FC 55          .01/01/92..U
FFFF:0010  60 10 00 F0 08 00 70 00-08 00 70 00 08 00 70 00  .p.p.p.
FFFF:0020  08 00 70 00 60 10 00 F0-60 10 00 F0 60 10 00 F0  ..p. ....
FFFF:0030  A5 FE 00 F0 87 E9 00 F0-55 FF 00 F0 60 10 00 F0  .....U....
FFFF:0040  60 10 00 F0 60 10 00 F0-80 10 00 F0 60 10 00 F0  .....
FFFF:0050  00 13 00 F0 00 11 00 F0-20 11 00 F0 40 11 00 F0  .....e...
FFFF:0060  A0 11 00 F0 C0 11 00 F0-E0 11 00 F0 20 12 00 F0  .....
FFFF:0070  C0 12 00 F0 C0 12 00 F0-40 12 00 F0 60 10 00 F0  .....e...
FFFF:0080  60 12 00 F0                                     ...
-d ffff:0004 001c
FFFF:0000          F0 30 31 2F-30 31 2F 39 32 00 FC 55          .01/01/92..U
FFFF:0010  60 10 00 F0 08 00 70 00-08 00 70 00 08          .....p.p..
-S_

```